

Visão Geral do Projeto Final

Tempo estimado: 30 minutos

Índice

- Introdução
 - Objetivos
- Visão Geral Detalhada
 - Etapa 1: Carregamento e Pré-processamento de Dados
 - Etapa 2: Modelo de Regressão Logística
 - Etapa 3: Treinamento do Modelo
 - Etapa 4: Otimização e Avaliação do Modelo
 - Etapa 5: Visualização e Interpretação
 - Etapa 6: Salvamento e Carregamento do Modelo
 - Etapa 7: Ajuste de Hiperparâmetros
 - Etapa 8: Importância das Variáveis

Introdução:

Neste projeto final, você embarcará em uma jornada empolgante para construir um modelo de regressão logística com o objetivo de prever os resultados das partidas de League of Legends. Aproveitando várias estatísticas do jogo, este projeto utilizará seu conhecimento de PyTorch, regressão logística e manipulação de dados para criar um modelo preditivo robusto. League of Legends, um popular jogo de arena de batalha multiplayer online (MOBA), gera uma extensa quantidade de dados a partir das partidas, oferecendo uma excelente oportunidade para aplicar técnicas de aprendizado de máquina em cenários do mundo real.

Objetivos:

- Carregar e pré-processar o conjunto de dados: Compreender e preparar os dados para o treinamento do modelo.
- Implementar e treinar um modelo de regressão logística: Desenvolver um modelo para prever os resultados das partidas.
- Avaliar o desempenho do modelo usando métricas apropriadas: Usar várias métricas para avaliar a precisão e a confiabilidade do modelo.
- Otimizar o modelo usando descida de gradiente e outras técnicas: Melhorar o desempenho do modelo por meio de otimização.
- Interpretar e visualizar os resultados: Obter insights a partir das previsões do modelo por meio da visualização.
- Salvar e carregar o modelo treinado: Aprender técnicas para persistir e recarregar modelos.
- Realizar ajuste de hiperparâmetros: Ajustar o modelo para um desempenho ideal.

Visão Geral Detalhada:

1. Passo 1: Carregamento e Pré-processamento de Dados
 - Tarefa 1: Carregar o conjunto de dados de League of Legends e pré-processá-lo para treinamento.

Este passo envolve a leitura dos dados, a divisão em conjuntos de treinamento e teste, e a padronização das características. Você irá preparar os dados para o treinamento do modelo utilizando pandas para manipulação de dados, `train_test_split` do sklearn para divisão de dados e `StandardScaler` para escalonamento de características. Os dados são então convertidos em tensores PyTorch, que são essenciais para o treinamento de modelos baseados em PyTorch.

2. Passo 2: Modelo de Regressão Logística
 - Tarefa 2: Implementar um modelo de regressão logística usando PyTorch.

Aqui, você definirá o modelo de regressão logística especificando as dimensões de entrada e a passagem direta utilizando a função de ativação sigmoid. Você preparará o terreno para o treinamento do modelo de regressão logística inicializando o modelo, a função de perda e o otimizador.

3. Passo 3: Treinamento do Modelo
 - Tarefa 3: Treinar o modelo de regressão logística no conjunto de dados.

O loop de treinamento será executado por um número especificado de épocas. Em cada época, o modelo faz previsões, calcula a perda, realiza a retropropagação e atualiza os parâmetros do modelo. Este processo iterativo ajuda a otimizar o modelo para prever com precisão os resultados das partidas.

4. Passo 4: Otimização e Avaliação do Modelo
 - Tarefa 4: Implementar técnicas de otimização e avaliar o desempenho do modelo.

Técnicas de otimização, como regularização L2 (Regras de Ridge), ajudam a prevenir o overfitting. O modelo é re-treinado com essas otimizações, e seu desempenho é avaliado em ambos os conjuntos de treinamento e teste para garantir robustez.

5. Passo 5: Visualização e Interpretação
 - Tarefa 5: Visualizar o desempenho do modelo e interpretar os resultados.

Ferramentas de visualização, como matrizes de confusão e curvas ROC, fornecem insights sobre o desempenho do modelo. A matriz de confusão ajuda a entender a precisão da classificação, enquanto a curva ROC ilustra a troca entre sensibilidade e especificidade.

6. Passo 6: Salvamento e Carregamento do Modelo
 - Tarefa 6: Salvar e carregar o modelo treinado.

Esta tarefa demonstra as técnicas para persistir um modelo treinado usando `torch.save` e recarregá-lo usando `torch.load`. Avaliar o modelo carregado garante que ele mantenha seu desempenho, tornando-o prático para implantação em aplicações do mundo real.

7. Passo 7: Ajuste de Hiperparâmetros
 - Tarefa 7: Realizar ajuste de hiperparâmetros para encontrar a melhor taxa de aprendizado.

Testando diferentes taxas de aprendizado, você identificará a taxa ótima que fornece a melhor precisão no teste. Este ajuste é crucial para melhorar o desempenho do modelo.

8. Passo 8: Importância das Características
 - Tarefa 8: Avaliar a importância das características para entender o impacto de cada característica na previsão.

Compreender a importância das características ajuda a identificar quais estatísticas do jogo são mais influentes na previsão dos resultados das partidas. Este passo envolve a extração dos pesos da camada linear e sua visualização.

Autor

Raghul Ramesh



Skills Network